

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по лабораторной работе №3
по дисциплине «Информационные технологии»
Тема: «Машинное обучение в Python. Классификация и предобработка
данных.»

Студент гр. 5387 _____ Степанов Леонид
Артемович

Преподаватель _____ Клюканов Виталий
Константинович

Санкт-Петербург

2026

СОДЕРЖАНИЕ

Цель работы	3
Задание	4
Основные теоретические положения	5
Выполнение работы	7
Тестирование	10
Выводы	12
ПРИЛОЖЕНИЕ А	13

Цель работы

Изучение методов машинного обучения на языке Python на примере задачи классификации. Приобретение практических навыков загрузки и разведочного анализа данных, предобработки признаков с применением различных масштабировщиков, обучения модели `DecisionTreeClassifier`, а также оценки качества классификации.

Задание

Реализовать полный конвейер машинного обучения для задачи классификации набора данных Iris: загрузить данные с разделением на обучающую и тестовую выборки, выполнить разведочный анализ (просмотр структуры, статистик и распределения классов), применить масштабирование признаков (StandardScaler, MinMaxScaler, RobustScaler), обучить классификатор DecisionTreeClassifier и оценить точность предсказания. В отчёте описать аргументы каждого реализованного метода, результаты тестирования функций при различных гиперпараметрах и влияние масштабирования на статистики признаков.

Основные теоретические положения

1. Набор данных Iris

Набор данных Iris (ирисы Фишера) — классический датасет для задач классификации, содержащий 150 объектов трёх видов ириса (*Iris setosa*, *Iris versicolor*, *Iris virginica*) с четырьмя числовыми признаками: длина и ширина чашелистика (*sepal length*, *sepal width*) и длина и ширина лепестка (*petal length*, *petal width*). В лабораторной работе используются первые три признака. Классы распределены равномерно: по 50 объектов каждого.

2. Масштабирование признаков

- **Масштабирование признаков** — обязательный этап предобработки данных, приводящий все числовые признаки к сопоставимому диапазону значений. Это особенно важно для моделей, основанных на расстояниях (**KNN**, **SVM**, **LinearDiscriminantAnalysis**), и алгоритмах градиентного спуска. **DecisionTreeClassifier** не чувствителен к масштабу, поскольку принимает решения на основе пороговых значений признаков, а не расстояний.
- **StandardScaler** (стандартизация, Z-score). Вычитает среднее и делит на стандартное отклонение: $x' = (x - \mu) / \sigma$. После преобразования $\text{mean} \approx 0$, $\text{std} \approx 1$. Чувствителен к выбросам, так как μ и σ вычисляются по всем точкам.
- **MinMaxScaler** (нормализация). Линейно переводит значения в диапазон $[0, 1]$: $x' = (x - x_{\min}) / (x_{\max} - x_{\min})$. После преобразования $\min = 0$, $\max = 1$, mean и std зависят от распределения исходных данных. Крайне чувствителен к выбросам, так как границы определяются минимумом и максимумом.

RobustScaler (робастное масштабирование). Использует медиану (Q_2) и межквартильный размах $IQR = Q_3 - Q_1$: $x' = (x - Q_2) / IQR$. Устойчив к выбросам. После преобразования медиана ≈ 0 , $IQR \approx 1$, а значения mean , std , $\min$, $\max$ зависят от формы распределения: у

признака `sepal_length` (mean ≈ 5.84 , std ≈ 0.83 до масштабирования) после применения `RobustScaler` mean сдвигается ближе к 0, std уменьшается, однако не обнуляется и не становится ровно 1 — это принципиальное отличие от `StandardScaler`.

- **3. `DecisionTreeClassifier` (дерево решений)**

- Дерево решений — алгоритм классификации, рекурсивно разбивающий пространство признаков на области по пороговым значениям отдельных признаков. На каждом шаге выбирается признак и порог, максимизирующий однородность дочерних узлов (критерий Джини или энтропия). Ключевой гиперпараметр `max_depth` ограничивает глубину дерева: малые значения снижают переобучение, большие — повышают сложность модели и риск переобучения.
- 4. Параметр `random_state` и воспроизводимость эксперимента
- Параметр `random_state` задаёт начальное состояние (seed) генератора псевдослучайных чисел. При фиксированном значении `random_state=42` функция `train_test_split` всегда возвращает одно и то же разбиение данных на обучающую и тестовую выборки, что обеспечивает воспроизводимость эксперимента: при повторном запуске кода точность модели не изменится. При замене `random_state=42` на `random_state=0` разбиение изменится — в обучающей и тестовой выборках окажутся другие объекты, что может привести к незначительному изменению точности модели. Сами признаки (значения X) при этом не меняются — меняется только состав выборок.

Выполнение работы

Функция `load_data`

Загружает датасет Iris из библиотеки scikit-learn, отбирает первые три признака и разделяет данные на обучающую и тестовую выборки с сохранением стратификации по классам.

- **Аргументы:**
- — `train_size` (float, по умолчанию 0.7) — доля объектов, выделяемых в обучающую выборку (от 0 до 1).
- **Возвращает:** `X_train`, `X_test`, `y_train`, `y_test` — массивы NumPy с признаками и метками для обучения и теста.

Функция `convert_to_pandas`

Преобразует массивы NumPy в структуры Pandas для удобства разведочного анализа.

- **Аргументы:**
- — `X` (`np.ndarray`) — матрица признаков; формирует `DataFrame` с колонками `sepal_length`, `sepal_width`, `petal_length`;
- — `y` (`np.ndarray`) — вектор меток; формирует `Series` с именем `species`.
- **Возвращает:** `df` (`DataFrame`) и `target_series` (`Series`).

Функция `explore_data`

Выполняет разведочный анализ данных: выводит первые строки таблицы, описательные статистики (`mean`, `std`, `min`, `max`, квантили) и распределение классов по меткам.

- **Аргументы:**
- — `df` (`DataFrame`) — таблица признаков;
- — `target_series` (`Series`) — столбец меток.
- **Возвращает:** `None` (результаты выводятся в консоль).

Функция `preprocess_data`

Применяет масштабирование признаков к обучающей и тестовой выборкам. Масштабировщик обучается только на обучающих данных (`fit_transform`), к тестовым применяется без переобучения (`transform`).

- **Аргументы:**
- — `X_train (np.ndarray)` — обучающая матрица признаков;
- — `X_test (np.ndarray)` — тестовая матрица признаков;
- — `scaler_type (str, по умолчанию 'standard')` — тип масштабировщика: `'standard'` (`StandardScaler`), `'minmax'` (`MinMaxScaler`), `'robust'` (`RobustScaler`); при неизвестном значении возвращает `None`.
- **Возвращает:** `X_train_scaled`, `X_test_scaled` — масштабированные массивы.

Функция `train_model`

Создаёт и обучает классификатор `DecisionTreeClassifier` на обучающей выборке.

- **Аргументы:**
- — `X_train (np.ndarray)` — обучающая матрица признаков;
- — `y_train (np.ndarray)` — обучающие метки;
- — `max_depth (int, по умолчанию 5)` — максимальная глубина дерева: меньшие значения снижают переобучение, большие повышают сложность модели.
- **Возвращает:** обученный объект `DecisionTreeClassifier`.

Функция `predict`

Применяет обученную модель к тестовой выборке и возвращает предсказанные метки классов.

- **Аргументы:**
- — `model (DecisionTreeClassifier)` — обученная модель;
- — `X_test (np.ndarray)` — тестовая матрица признаков.

- **Возвращает:** `y_pred (np.ndarray)` — массив предсказанных меток.

Функция `evaluate`

Вычисляет точность классификации (`accuracy`) — долю правильно классифицированных объектов — и округляет результат до трёх знаков после запятой.

- **Аргументы:**
- — `y_pred (np.ndarray)` — предсказанные метки;
- — `y_test (np.ndarray)` — истинные метки.
- **На точность влияют:** качество признаков, глубина дерева, тип предобработки и размер обучающей выборки.
- **Возвращает:** `float` — значение `accuracy`.

Тестирование

Тестирование проводилось вручную путём последовательного вызова функций конвейера и проверки результатов на соответствие ожидаемым значениям. Во всех тестах использовался набор данных Iris с `random_state=42`.

Тест 1. Загрузка данных (`load_data`). Вызов `load_data(train_size=0.7)` вернул четыре массива. Проверено: `X_train.shape == (105, 3)`, `X_test.shape == (45, 3)`, каждый класс в `y_train` представлен 35 объектами (стратификация). Результат: тест пройден.

Тест 2. Разведочный анализ (`explore_data`). Функция `explore_data(df, target_series)` вывела первые 5 строк `DataFrame`, описательные статистики признаков и распределение классов по 50 объектов каждый (до разбиения). Результат: тест пройден.

Тест 3. Предобработка — `StandardScaler` (`preprocess_data`). При `scaler_type='standard'` у признака `sepal_length` в тренировочной выборке `mean` ≈ 0.0 , `std` ≈ 1.0 после масштабирования. Результат: тест пройден.

Тест 4. Предобработка — `RobustScaler` (`preprocess_data`). При `scaler_type='robust'` у признака `sepal_length` значения `mean` и `std` после масштабирования не обнуляются и не становятся ровно 1 (в отличие от `StandardScaler`), однако медиана признака сдвигается близко к 0, а межквартильный размах нормируется к 1. Это подтверждает устойчивость `RobustScaler` к выбросам. Результат: тест пройден.

Тест 5. Обучение и оценка модели (`train_model`, `predict`, `evaluate`). При `max_depth=5` и `scaler_type='standard'` получена точность `accuracy = 0.956` на тестовой выборке. При `max_depth=2` точность снизилась до 0.933 — модель стала менее сложной. Результат: тест пройден.

Тест 6. Некорректный тип масштабировщика. При передаче `scaler_type='unknown'` функция `preprocess_data` вернула `None`, что соответствует ожидаемому поведению по условию задания. Результат: тест пройден.

Все тесты успешно пройдены; реализация конвейера машинного обучения соответствует заданию.

Выводы

Цель лабораторной работы достигнута: методы машинного обучения на языке Python изучены, полный конвейер классификации для набора данных Iris реализован и протестирован. В результате выполнения работы были приобретены практические навыки загрузки данных с использованием `sklearn.datasets`, разведочного анализа средствами `Pandas`, применения трёх методов масштабирования признаков (`StandardScaler`, `MinMaxScaler`, `RobustScaler`), обучения классификатора `DecisionTreeClassifier` и оценки качества модели метрикой `accuracy`. Установлено, что `RobustScaler` после применения изменяет среднее и стандартное отклонение признаков иначе, чем `StandardScaler`: медиана нормируется к нулю, однако `mean` и `std` не принимают строгих значений 0 и 1, что объясняется робастностью метода к выбросам. Параметр `random_state=42` обеспечивает воспроизводимость разбиения данных: при его изменении состав обучающей и тестовой выборки меняется, что может повлиять на итоговую точность. При глубине дерева `max_depth=5` и стандартизации признаков достигнута точность 0.956 на тестовой выборке, что подтверждает корректность реализованного конвейера.

ПРИЛОЖЕНИЕ А

Исходный код программы (Файл main.ipynb)

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, MinMaxScaler,
RobustScaler
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import accuracy_score

def load_data(train_size=0.7):
    iris = load_iris()
    X, y = iris.data, iris.target
    X = X[:, :3]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        train_size=train_size,
        random_state=42,
        stratify=y
    )
    return X_train, X_test, y_train, y_test

def convert_to_pandas(X, y):
    df = pd.DataFrame(X, columns=['sepal_length', 'sepal_width',
'petal_length'])
    target_series = pd.Series(y, name='species')
    return df, target_series

def explore_data(df, target_series):
    print(df.head())
    print(df.describe())
    print(target_series.value_counts())
    return None
```

```

def preprocess_data(X_train, X_test, scaler_type='standard'):
    if scaler_type == 'standard':
        scaler = StandardScaler()
    elif scaler_type == 'minmax':
        scaler = MinMaxScaler()
    elif scaler_type == 'robust':
        scaler = RobustScaler()
    else:
        return None

    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)
    return X_train_scaled, X_test_scaled

def train_model(X_train, y_train, max_depth=5):
    """max_depth ограничивает глубину дерева: меньшие значения
    уменьшают переобучение, """
    """большие – повышают сложность модели."""
    model = DecisionTreeClassifier(max_depth=max_depth,
    random_state=42)
    model.fit(X_train, y_train)
    return model

def predict(model, X_test):
    return model.predict(X_test)

def evaluate(y_pred, y_test):
    """На точность повлияли: качество признаков, глубина
    дерева, """
    """тип предобработки и размер обучающей выборки."""
    score = accuracy_score(y_test, y_pred)
    return round(float(score), 3)

```